1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.
Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

Register R0

31–24	23–16	15–8	7–0
00	00	00	12

Register R1

31–24	23–16	15–8	7–0
ff	ff	ff	80

Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

Adresse der Konstanten:

31–24	23–16	15–8	7–0
80	00	02	30

Inhalt Register R2 nach Anw-03:

31–24	23–16	15–8	7–0
12	34	56	78

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

ldr r2, [pc, #56]

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss
Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

Adresse v. VariableA (Reg. R0 nach Anw-05)

31–24	23–16	15–8	7–0
20	00	00	0c

Inhalt Register R1 nach Anw-05:

31–24	23–16	15–8	7–0
00	00	12	34

Anzahl von Anw-05 geladener Bits: 16

Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:

31–24	23–16	15–8	7–0
47	11	12	39

Anzahl von Anw-06 geladener Bits: 32

Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

Adresse v. VariableC:

31–24	23–16	15–8	7–0
20	00	00	10

Inhalt von VariableC nach Anw-07:

31–24	23–16	15–8	7–0
34	12	11	47

Anzahl von Anw-07 gespeicherter Bits: 32

3 Zugriff auf Felder

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

	Anfangsadresse:	Anzahl Elemente:	Anzahl Bytes pro Element:
MeinHalbwortFeld:	20 00 00 14 (31 … 16 15 … 0)	6	2
MeinWortFeld:	20 00 00 20 (31 … 16 15 … 0)	6	4
MeinTextFeld:	20 00 00 38 (31 … 16 15 … 0)	2	1

Anw-08 *Anw-09* *Anw-10* Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für das jeweilige Element erfolgen.

Register R1 nach Anw-09:	Register R2 nach Anw-10:
00 00 00 22 (31 … 16 15 … 0)	00 00 00 3e (31 … 16 15 … 0)

Anw-11 *Anw-12* Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:

Register R4 nach Anw-12:	Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements:
00 00 00 45 (31 … 16 15 … 0)	5

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13	Register R5 nach Anw-13:	Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements:
20 00 00 16 (31 … 16 15 … 0)	00 00 00 3e (31 … 16 15 … 0)	1

Anw-14

Register R0 nach Anw-14	Register R6 nach Anw-14:	Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements:
20 00 00 18 (31 … 16 15 … 0)	00 00 ff cc (31 … 16 15 … 0)	2

Anw-15

Register R0 nach Anw-15	Adresse des von Anw-15 überschriebenen Elements:	Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements:
20 00 00 1a (31 … 16 15 … 0)	20 00 00 1a (31 … 16 15 … 0)	3

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Anw-16
Anw-17
Anw-18
Anw-19

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Reg. R1:	00010010 00210100 01010210 01211000	305419896	305419896
Reg. R2:	10011101 11001010 01011001 10000110	-1647683194	2647284102
Reg. R3:	1010 1111 1111 1110 1010 1111 1111 1110	-1342263298	2952658994

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ~~N~~ Z

Ergebnis: ~~Ok~~ Nicht Ok — **Ergebnis:** ~~Ok~~ Nicht Ok

Anw-20
Anw-21
Anw-22

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Reg. R4:	11001100 00000000 00000000 00000000	-872415232	3422552064
Reg. R5:	01001110 00000000 00000000 00000000	1308622848	1308622848
Reg. R6:	00000000 00000000 11111111 11001100	65484	65484

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: ~~V~~ C N Z

Ergebnis: Ok ~~Nicht Ok~~ — **Ergebnis:** Ok ~~Nicht Ok~~

Anw-23
Anw-24
Anw-25

		dezimale Interpretation der Bitmuster signed	dezimale Interpretation der Bitmuster unsigned
Reg. R7:	00100111 00000000 00000000 00000000	654311424	654311424
Reg. R8:	01000101 00000000 00000000 00000000	1157627904	1157627904
Reg. R9:	11100010 00000000 00000000 00000000	-503316480	3791650816

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ~~N~~ Z

Ergebnis: ~~Ok~~ Nicht Ok — **Ergebnis:** Ok ~~Nicht Ok~~